

STARVNT VENDOR DASHBOARD V1

DAY 4 + DAY 5 MASTER EXECUTION PLAN

Real Business Flow | Security | Reliability | Automation | Working Proof

DAY 4 - BUILD THE REAL BUSINESS FLOW.

DAY 5 - BREAK THE BUSINESS FLOW AND PROVE IT.

DOCUMENT MAP

What this phase is designed to accomplish

- Close the gap between isolated modules and one connected Vendor business flow.
- Use real API and database data after integration begins; eliminate fake dashboard output from final demos.
- Force direct frontend-backend collaboration between Nesar and Yogesh.
- Use Day 4 to establish the real enquiry operating flow.
- Use Day 5 to attack that flow with security, duplication, failure, retry, and audit tests.
- Finish with a combined engineering report backed by repository, commit, API, database, and UI proof.

Engineering north star

Simple for the user. Powerful underneath. Secure by design. Proven by reality.

CONTENTS

No.	Section
1	Mandatory Reality Check
2	Day 4 - Real Vendor Enquiry Operating Flow
3	Yogesh - Day 4 Backend Ownership
4	Nesar - Day 4 Frontend Ownership
5	Joint Integration Contract
6	Day 4 Golden Demo and Acceptance
7	Day 5 - Security and Reliability Hardening
8	Yogesh - Day 5 Backend Ownership
9	Nesar - Day 5 Frontend Ownership
10	Day 5 Stress Tests
11	Combined End-to-End Demo
12	Combined Final Report
13	CEO Acceptance Standard
14	Strategic Position and Day 6 Direction

1. MANDATORY REALITY CHECK

Before Day 4 starts, the team must verify the actual implementation status of Day 1-3. Do not assume that code, folders, documentation, or screenshots mean a component is complete.

The principle is:

Assumption is not architecture. Proof is architecture.

Component	Status	Required Proof
Frontend	Working / Partial / Broken	Running UI / Screenshot
Backend	Working / Partial / Broken	API Test
Database	Connected / Not Connected	Connection Proof
Authentication	Working / Partial / Broken	Login Test
Vendor Dashboard	Working / Partial / Broken	UI Proof
Existing APIs	Working / Partial / Broken	Endpoint Test
GitHub	Updated / Outdated	Latest Commit Hash
Frontend -> Backend	Connected / Not Connected	Real Working Flow

Required submission

```
Repository:
Frontend Branch:
Backend Branch:

Latest Frontend Commit:
Latest Backend Commit:

Frontend Run Command:
Backend Run Command:

Database:
Environment:

Current Working Flow:
Current Blockers:
```

No new architecture before understanding the existing architecture.

Execution principle: REVIEW -> REUSE -> EXTEND. Do not ignore a working foundation and create a competing implementation.

2. DAY 4 - REAL VENDOR ENQUIRY OPERATING FLOW

Day 4 is the first serious integration phase for the Vendor Dashboard. The goal is to establish one real, end-to-end business operation rather than a collection of disconnected components.

Golden flow

```
USER
|
v
CREATE ENQUIRY
|
v
BACKEND VALIDATION
|
v
DATABASE SAVE
|
v
BUSINESS EVENT
|
v
AUTOMATION RULE
|
v
CONDITION CHECK
|
v
ACTIVITY CREATED
|
v
NOTIFICATION CREATED
|
v
VENDOR DASHBOARD UPDATED
```

Primary rule

No fake data after integration begins. Frontend mock data may be used during isolated UI work, but the final Day 4 flow must use real backend responses and real database state.

- No hardcoded enquiry cards in the final demo.
- No fake dashboard metrics.
- No static notification objects.
- No dummy activity timeline.
- No frontend-only success state without backend confirmation.

3. YOGESH - DAY 4 BACKEND OWNERSHIP

Yogesh owns backend correctness and the operational business flow. The frontend depends on a stable, predictable contract from the backend.

3.1 Enquiry domain model

```
enquiryId
organizationId
vendorId
customerId

name
email
phone

eventType
eventDate
location
```

```
budget
message

status
priority
source
assignedTo

createdAt
updatedAt
```

Critical ownership rule

```
AUTHENTICATED USER
|
v
USER CONTEXT
|
v
VENDOR / ORGANIZATION RESOLVED
|
v
SERVER ATTACHES OWNERSHIP
|
v
SAVE ENQUIRY
```

The backend must not blindly trust frontend-controlled ownership fields such as organizationId, vendorId, or createdBy.

3.2 Status system

```
NEW -> CONTACTED -> QUALIFIED -> PROPOSAL_SENT -> WON
NEW / CONTACTED / QUALIFIED -> LOST

WON -> CLOSED
LOST -> CLOSED
```

State transition protection

- Use centralized transition validation.
- Do not permit arbitrary status jumps unless the business rule explicitly allows them.
- Examples: NEW -> CONTACTED is valid; CONTACTED -> QUALIFIED is valid; CLOSED -> NEW is not valid by default.

3.3 Required APIs

```
POST /api/enquiries
GET /api/enquiries
GET /api/enquiries/:id
PUT /api/enquiries/:id
PATCH /api/enquiries/:id/status
DELETE /api/enquiries/:id
```

> **Deletion is not automatically required.**
>
> If business requirements do not need DELETE yet, do not build it simply to satisfy CRUD symmetry. Archive or soft-delete may be safer later.

3.4 API response consistency

```
SUCCESS
{
"success": true,
"data": {},
"message": "Enquiry created successfully"
}

FAILURE
{
"success": false,
"code": "VALIDATION_ERROR",
"message": "Invalid enquiry data"
}
```

Recommended error codes

VALIDATION_ERROR | UNAUTHORIZED | FORBIDDEN | NOT_FOUND | INVALID_STATE | DUPLICATE_RESOURCE | INTERNAL_ERROR

3.5 Business events

Required Day 4 events: ENQUIRY_CREATED, ENQUIRY_STATUS_CHANGED, ENQUIRY_WON, ENQUIRY_LOST.

Event contract

```
eventId
eventName
organizationId
entityType
entityId
actorId
data
timestamp
```

3.6 Automation rule

```
RULE: HIGH_PRIORITY_ENQUIRY_ALERT
TRIGGER: ENQUIRY_CREATED
CONDITION: priority = HIGH OR priority = URGENT
ACTIONS: CREATE_ACTIVITY + CREATE_NOTIFICATION
```

If the condition fails, the automation should SKIP. A skipped rule is not an error.

3.7 Activity API and Notification API

```
GET /api/enquiries/:id/activities
GET /api/notifications
PATCH /api/notifications/:id/read
```

3.8 Backend Day 4 success proof

```
POST /api/enquiries
|
v
ENQUIRY SAVED
|
v
ENQUIRY_CREATED
|
v
RULE MATCH
|
v
CONDITION TRUE
|
```

```
v
ACTIVITY + NOTIFICATION SAVED
```

4. NESAR - DAY 4 FRONTEND OWNERSHIP

Nesar owns the operational experience. The UI must make the business state obvious and must consume real backend data.

4.1 Vendor dashboard

Required metrics: TOTAL ENQUIRIES, NEW, HIGH PRIORITY, QUALIFIED, WON, LOST. Metrics must come from APIs.

4.2 Enquiry list

Columns: Customer, Event Type, Event Date, Budget, Priority, Status, Created.

Required UI states

- Loading: Loading enquiries...
- Empty: No enquiries yet.
- Error: Unable to load enquiries.
- Recovery: Retry

4.3 Enquiry details

Display Customer, Event, Budget, Message, Priority, Status, Assigned To, and a real timeline.

Example timeline

```
ENQUIRY CREATED
HIGH PRIORITY ACTIVITY CREATED
STATUS CHANGED
NOTIFICATION GENERATED
```

4.4 Status update flow

```
FRONTEND
|
v
PATCH REQUEST
|
v
BACKEND VALIDATION
|
v
STATE TRANSITION VALIDATION
|
v
DATABASE UPDATE
|
v
BUSINESS EVENT
|
v
ACTIVITY / AUDIT
|
v
FRONTEND REFRESH
```

The frontend must display the final backend result. It must not show an optimistic success state that contradicts server state.

5. JOINT INTEGRATION CONTRACT

Before integration, Yogesh provides the backend contract and Nesar confirms the frontend behavior.

Yogesh must provide

- API Base URL
- Endpoint and HTTP method
- Request body
- Success response
- Error response
- Authentication requirement

Nesar must confirm

- API connected
- Loading state works
- Success state works
- Error state works
- Real data is displayed
- Related entity navigation works where applicable

Developer communication rule

```
NESAR <-> YOGESH
|
v
DIRECT INTEGRATION
|
v
CEO RECEIVES PROGRESS + BLOCKERS
```

The CEO should not become an API messenger. Developers should solve routine integration issues directly and escalate decisions or blockers.

6. DAY 4 GOLDEN DEMO AND ACCEPTANCE

The final Day 4 demo must run on the real product path.

```
OPEN DASHBOARD
|
v
CREATE HIGH PRIORITY ENQUIRY
|
v
VALIDATE INPUT
|
v
SAVE DATABASE
|
v
CREATE ENQUIRY_CREATED EVENT
|
v
AUTOMATION RULE MATCH
|
v
PRIORITY CONDITION TRUE
```

```
|
v
CREATE ACTIVITY
|
v
CREATE NOTIFICATION
|
v
DASHBOARD REFRESH
|
v
OPEN ENQUIRY DETAILS
|
v
VIEW REAL TIMELINE
```

Day 4 acceptance

Backend

- Create Enquiry
- List Enquiries
- Get Details
- Update Status
- Business Event
- Automation Rule
- Condition Evaluation
- Activity persistence
- Notification persistence

Frontend

- Real Dashboard
- Real Enquiry List
- Real Details
- Status Update
- Activity Timeline
- Notifications

Integration

- Frontend -> Backend
- Backend -> Database
- Event -> Automation
- Automation -> Activity
- Automation -> Notification

7. DAY 5 - SECURITY + RELIABILITY + PRODUCTION HARDENING

Day 4 asks: Does the system work? Day 5 asks: What happens when the system is misused, repeated, attacked, or partially fails?

Day 5 principle

Break the business flow and prove it can fail safely, deny unauthorized access, avoid duplicate actions, and recover when possible.

8. YOGESH - DAY 5 BACKEND OWNERSHIP

8.1 Authentication testing

```
NO TOKEN
|
v
API REQUEST
|
v
401 UNAUTHORIZED
```

Also test invalid or expired authentication according to the existing authentication implementation.

8.2 Authorization

```
USER
|
v
REQUEST ACTION
|
v
PERMISSION CHECK
|
v
ALLOW / DENY
```

Do not fake an enterprise RBAC layer.

If a reusable RBAC system does not yet exist, implement the minimum correct enforcement required for the Vendor Dashboard and document the future expansion scope.

8.3 Vendor / organization isolation

```
VENDOR A
|
v
CREATES ENQUIRY A

VENDOR B
|
v
GET /api/enquiries/A
|
v
403 FORBIDDEN OR 404 NOT FOUND
```

The exact denial response can depend on the chosen security strategy. The non-negotiable rule is: Vendor A data must never be returned to Vendor B.

8.4 Tenant-aware query principle

```
BAD: id = enquiryId

CORRECT: id = enquiryId
AND organizationId = authenticatedOrganizationId
```

8.5 Duplicate protection and idempotency

```
EVENT_123
|
v
ACTIVITY CREATED

EVENT_123 AGAIN
|
v
ALREADY PROCESSED?
|
+-- YES -> SKIP
```

Suggested idempotency key: eventId + actionType. Example: EVT_123_CREATE_ACTIVITY.

8.6 Failure handling

```
ENQUIRY CREATED [OK]
EVENT CREATED [OK]
AUTOMATION STARTED [OK]
NOTIFICATION ACTION [FAIL]
|
v
CAPTURE -> LOG -> RETRY -> SUCCESS / FINAL FAILURE
```

8.7 Retry policy

```
ATTEMPT 1 -> FAIL
ATTEMPT 2 -> FAIL
ATTEMPT 3 -> SUCCESS

Suggested V1: maxRetries = 3
```

Keep retry configuration centralized. Do not duplicate magic numbers across files.

8.8 Audit trail

```
actorId
organizationId
entityType
entityId
action
before
after
timestamp
```

Audit must answer: Who changed it? What changed? When? From what? To what?

9. NESAR - DAY 5 FRONTEND OWNERSHIP

9.1 Error UX

- API failure: Unable to load data.
- Recovery: Retry
- Unauthorized: Please login again.
- Forbidden: You do not have permission to access this resource.
- Empty: No enquiries found.
- Loading: Loading...

Never leave the user with an unexplained blank screen or a false success state.

9.2 Real notification center

```
NOTIFICATION BELL: [3]
|
v
NOTIFICATIONS
|
+-- High Priority Enquiry
+-- Wedding Event
+-- Budget 500000
+-- Just now
```

Required: unread count, notification list, mark read, navigate to the related enquiry. All data comes from backend APIs.

9.3 Dashboard refresh

After Create Enquiry and after Status -> WON, the dashboard should reflect updated metrics. API refetch is acceptable for V1. Do not claim real-time unless WebSocket/SSE is actually implemented.

10. DAY 5 STRESS TESTS

TEST 1 - Normal flow

```
LOGIN
CREATE HIGH PRIORITY ENQUIRY
DATABASE
EVENT
AUTOMATION
ACTIVITY
NOTIFICATION
DASHBOARD

Expected: SUCCESS
```

TEST 2 - Cross-vendor access

```
VENDOR B
REQUEST VENDOR A ENQUIRY

Expected: DENIED
```

TEST 3 - Duplicate event

```
SAME EVENT AGAIN

Expected: NO DUPLICATE ACTIVITY + NO DUPLICATE NOTIFICATION
```

TEST 4 - Automation failure

```
ACTION FAILS

Expected: ERROR CAPTURED -> RETRY -> SUCCESS OR FINAL FAILURE
```

TEST 5 - Invalid status

```
CLOSED
NEW

Expected: INVALID_STATE
```

11. COMBINED DAY 4 + DAY 5 END-TO-END DEMONSTRATION

This is the proof that the two-day phase produced a real connected system rather than separate modules.

```
USER LOGIN
|
v
AUTH VERIFIED
|
v
VENDOR / ORGANIZATION CONTEXT
|
v
CREATE HIGH PRIORITY ENQUIRY
|
v
BACKEND VALIDATION
|
v
DATABASE SAVE
|
v
ENQUIRY_CREATED EVENT
|
v
AUTOMATION RULE MATCH
|
v
CONDITION CHECK
|
v
ACTIVITY CREATED
|
v
NOTIFICATION CREATED
|
v
DASHBOARD UPDATED
|
v
OPEN ENQUIRY
|
v
CHANGE STATUS
|
v
STATUS VALIDATED
|
v
DATABASE UPDATED
|
v
STATUS EVENT
|
v
AUDIT LOG CREATED
|
v
TIMELINE UPDATED
```

Then demonstrate the hard cases:

```
VENDOR B -> TRY TO ACCESS VENDOR A DATA -> BLOCKED
SAME EVENT AGAIN -> IDEMPOTENCY CHECK -> SKIPPED
AUTOMATION ACTION FAILS -> ERROR CAPTURED -> RETRY -> RECOVERY / FINAL FAILURE
```

12. COMBINED DAY 4-5 FINAL REPORT

Both developers must submit one combined report rather than two disconnected reports.

REPOSITORY

Repository:
Frontend Branch:
Frontend Commit:
Backend Branch:
Backend Commit:

FOUNDATION

Frontend:
Backend:
Database:
Authentication:
API Integration:

ENQUIRIES

Create:
List:
Details:
Update:
Status Update:

BUSINESS EVENTS

ENQUIRY_CREATED:
ENQUIRY_STATUS_CHANGED:
ENQUIRY_WON:
ENQUIRY_LOST:

AUTOMATION

Rule:
Trigger:
Condition:
Actions:
Working Proof:

ACTIVITY

Created:
Stored:
API:
Timeline Display:

NOTIFICATIONS

Created:
Stored:
API:
Unread Count:
Mark Read:
Navigation:

SECURITY

401 Tested:
403 Tested:
Cross-Vendor Access:
Tenant / Organization Isolation:

RELIABILITY

Duplicate Protection:
Idempotency:
Failure Handling:
Retry:
Final Failure Logging:

AUDIT

Actor:
Action:
Before:
After:
Timestamp:

FRONTEND UX

Loading:
Empty:
Error:
Retry:
Unauthorized:
Forbidden:

FINAL GOLDEN FLOW

WORKING / PARTIAL / NOT WORKING
Explain honestly.

KNOWN BUGS

1.
2.
3.

BLOCKERS

1.
2.

DAY 6 RECOMMENDATION

1.
2.

13. CEO ACCEPTANCE STANDARD

Completion is determined by proof, not by statements.

Developer says: Backend complete. Question: Has a real request successfully come from the frontend?
Developer says: Automation complete. Question: Has a real business event triggered it?
Developer says: Notification complete. Question: Was it stored in the database and displayed in the UI?

Developer says: Security complete.

Question: Was a cross-vendor access attack actually tested?

Developer says: Retry complete.

Question: Was an intentional failure created and the retry demonstrated?

A feature is not complete because a UI exists, an API exists, a model exists, code compiles, or one Postman call returned 200. The operational standard is:

```
IMPLEMENTED
|
INTEGRATED
|
SECURED
|
VALIDATED
|
TESTED
|
FAILURE TESTED
|
AUDITABLE
|
OBSERVABLE
|
DEMONSTRATED
|
PROVEN
```

14. STRATEGIC POSITION AND DAY 6 DIRECTION

The strongest outcome of this two-day phase is not feature count. It is one complete business flow that is real, connected, secure, traceable, tested, and difficult to break.

StarVnt should avoid spending this stage on:

- More disconnected dashboards
- Random UI pages
- Fancy animations
- Dozens of APIs with no end-to-end consumer
- Parallel or duplicate modules

The operating architecture after Day 5 should look like:

```
IDENTITY
|
v
VENDOR ACCESS
|
v
ENQUIRY
|
v
DATABASE
```

```
|
v
BUSINESS EVENT
|
v
AUTOMATION
|
v
ACTIVITY
|
v
NOTIFICATION
|
v
DASHBOARD
|
v
AUDIT
|
v
SECURITY
|
v
RELIABILITY
```

Natural Day 6 direction

```
ENQUIRY
|
v
FOLLOW-UP
|
v
ASSIGNMENT
|
v
QUOTATION
|
v
BOOKING
|
v
PAYMENT
|
v
CUSTOMER JOURNEY
|
v
AURA+ INSIGHT
```

Final principle

DAY 4 BUILDS THE ENGINE. DAY 5 PROVES THE ENGINE DOES NOT FALL APART WHEN REALITY HITS IT.

StarVnt should be simple for the user, powerful underneath, secure by design, and proven by reality.

OFFICIAL ENGINEERING STANDARD

REVIEW -> REUSE -> EXTEND -> CONNECT -> SECURE -> TEST -> BREAK -> RECOVER -> DEMONSTRATE

SIMPLE FOR THE USER. RUTHLESSLY STRONG UNDERNEATH.